



CENTRIFUGE INTEGRATION

CentrifugeFacet v1.0.0, ICentrifugeTokenBridge v1.0.0

Independent Security Review

SECURITY RESEARCHER

Sujith Somraaj
(somraajsujith@gmail.com)

REPORT PREPARED BY

Sujith Somraaj

Report issued September 23, 2026

Table of Contents

Table of Contents	2
Executive Summary	3
About the Researcher	4
Disclaimer	5
Scope and Repository	6
Risk Classification	7
Severity Guidance	7
Findings	8
Low Risk	8
Informational	9

Executive Summary

Over 10 review hours, LI.FI engaged Sujith Somraaj to perform a differential security review of CentrifugeFacet v1.0.0 and ICentrifugeTokenBridge v1.0.0 at commit [9b3596f](#).

TOTAL FINDINGS	REVIEW DURATION	RESOLUTION STATUS
6 2 Low Risk · 4 Informational	10h September 21, 2026	2 / 4 2 fixed · 4 acknowledged

Project	CentrifugeFacet v1.0.0, ICentrifugeTokenBridge v1.0.0
Project type	Solidity Smart Contracts
Repository	lifinance/contracts
Pull request	PR #2322
Methods	Manual Review
Documentation	High
Testing coverage	High

About the Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the Web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over \$170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast, Angles and multiple other protocols.

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

Scope and Repository

Contract	Path and version
CentrifugeFacet	src/Facets/CentrifugeFacet.sol (v1.0.0)
ICentrifugeTokenBridge	src/Interfaces/ICentrifugeTokenBridge.sol (v1.0.0)

Repository	Commit
contracts	9b3596f

This was a differential review limited to changes introduced by the audit commit.

Risk Classification

Likelihood / Impact	High impact	Medium impact	Low impact
High likelihood	Critical	High	Medium
Medium likelihood	High	Medium	Low
Low likelihood	Medium	Low	Low

Severity Guidance

Critical: Must fix as soon as possible. **High:** Must fix before deployment. **Medium:** Should fix. **Low:** Could fix. Informational findings document integration assumptions, quality concerns, or operational risks.

Findings

Low Risk

Destination eligibility is not validated before dispatch

Context: CentrifugeFacet.sol#L145-L170, CentrifugeFacet.sol#L194-L200

Description: The facet can only validate the source-side transfer. It does not ensure that the share token is registered on the destination or that the receiver may hold a restricted share token. The source transaction can succeed while destination execution fails and requires manual retry.

Recommendation: Restrict v1 to explicitly supported token/chain pairs and have the backend verify receiver eligibility before quoting. Document and monitor the manual retry process for messages that still fail on the destination.

LI.FI: Acknowledged. The destination chain is validated on-chain (TokenBridge.send reverts InvalidChainId() unless chainIdToCentrifugeId is set; only Ethereum and Base are mapped), and beyond that no integrator-level check is possible on-chain. Backend will do its verification before calldata is prepared.

Researcher: Acknowledged.

Positive-slippage documentation is incorrect

Context: CentrifugeFacet.sol#L44-L47, CentrifugeFacet.sol#L132-L140, CentrifugeFacet.sol#L194-L200

Description: The CentrifugeData.refundRecipient NatSpec says it receives positive slippage. In practice, _depositAndSwap assigns the full swap output to _bridgeData.minAmount, and _startBridge sends that full amount to the destination receiver.

Recommendation: Update the NatSpec to state that positive slippage is bridged to _bridgeData.receiver. If refunds are intended instead, bridge the quoted amount and explicitly return the surplus.

LI.FI: Fixed in 4cfb7ab

Researcher: Verified fix.

Informational

Swap path can spend and refund the Diamond's existing ERC20 balances

Context: CentrifugeFacet.sol#L132-L140, LibAsset.sol#L132-L141, SwapperV2.sol#L336-L353

Description: The swap path trusts the caller-supplied `requiresDeposit` flag. When it is false, no tokens are pulled from the caller, yet the whitelisted router can spend the Diamond's existing input-token balance. Even when `requiresDeposit` is true, leftover handling sends the input token's entire remaining balance not just this call's balance delta to the caller-controlled `refundRecipient`.

Recommendation: Fix SwapperV2 centrally: snapshot each distinct input asset before deposits, require swaps to preserve that baseline, and refund only the positive per-call delta. For native assets, preserve both the initial balance and `nativeReserve`.

LI.FI: Acknowledged - as your recommendation notes, the fix belongs in SwapperV2 centrally, which is inherited by ~30 facets and out of scope for this PR.

Researcher: Acknowledged.

External bridge configuration can change after deployment

Context: CentrifugeFacet.sol#L32-L33, CentrifugeFacet.sol#L194-L200

Description: `TOKEN_BRIDGE` is immutable, but Centrifuge's routing, Spoke, Gateway, and relayer configuration remains externally controlled. Those changes can alter supported routes and refund behavior without changing the address stored by the facet.

Recommendation: Monitor the relevant Centrifuge configuration and admin events on Ethereum and Base. Alert or pause quoting when expected routes, dependencies, or refund settings change.

LI.FI: Acknowledged - this is handled by monitoring rather than in the facet.

Researcher: Acknowledged.

Oversized amounts revert only in the external bridge

Context: ICentrifugeTokenBridge.sol#L26-L32, CentrifugeFacet.sol#L194-L200

Description: The facet accepts a `uint256` amount, while Centrifuge later checked-casts the amount to `uint128`. Values above `type(uint128).max` fail only after entering the external bridge. The transaction is atomic, so funds are not lost, but the failure is late and unclear.

Recommendation: Reject amounts above `type(uint128).max` locally with a clear custom error, or explicitly document the upstream limit.

LI.FI: Documented in 1e85b1f.

Researcher: Added documentation looks good. Acknowledged.

Swap path can spend the Diamond's existing native balance

Context: CentrifugeFacet.sol#L129-L140, CentrifugeFacet.sol#L194-L200, SwapperV2.sol#L67-L77

Description: `swapAndStartBridgeTokensViaCentrifuge` reserves `nativeFee` during swaps but never proves that the fee came from this call. `_startBridge` therefore can pay the fee from unrelated native funds already held by the Diamond. Centrifuge then refunds any fee surplus to the caller-controlled `refundRecipient`, making most of that balance extractable.

For example,

- 1 The Diamond holds 10 ETH from an unrelated transfer.
- 2 An attacker performs an ERC20-only swap into a valid share token with `msg.value = 0`, `nativeFee = 10 ETH`, and their address as `refundRecipient`.
- 3 The facet sends the Diamond's 10 ETH to TokenBridge.
- 4 If the actual messaging cost is `C`, the Gateway refunds `10 ETH - C` to the attacker.
- 5 `refundExcessNative` sees a final balance below its initial snapshot and refunds nothing. The attacker receives `10 ETH - C`, while the Diamond also pays `C` for the attacker's bridge.

Recommendation: Snapshot `address(this).balance - msg.value` before the swaps and require the post-swap balance increase to cover `nativeFee`. This still permits ERC20-to-native fee swaps while preventing use of pre-existing Diamond funds. Add a regression test with a pre-funded Diamond, `msg.value = 0`, and no native-producing swap.

LI.FI: Acknowledged. The Diamond is not designed to custody funds, so any balance left in it is already treated as lost; the same swap-path shape exists in other facets and closing it here alone would diverge without closing the class.

Researcher: Acknowledged.